LAPORAN PRAKTIKUM

ALGORITMA DAN STRUKTUR DATA

Disusun Oleh:

Nama : **M. SYAUQI AUFA IZZAZULLAH**

NIM : **2025573010075**

Kelas : **1D – TI**

Prodi : **Teknologi Informasi Dan Komputer**

Pembimbing : **M. REZA ZULMAN, S.ST., M.SC**

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE 2026

Kata pengantar

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya sehingga laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban penulis setelah melaksanakan kegiatan praktikum.

Penulis menyadari bahwa dalam penyusunan laporan praktikum ini masih terdapat berbagai kekurangan. Namun berkat bantuan, bimbingan, serta dukungan dari berbagai pihak, laporan ini dapat diselesaikan. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan terima kasih kepada dosen pengampu mata praktikum serta semua pihak yang telah membantu dalam pelaksanaan praktikum hingga penyusunan laporan ini.

Penulis berharap laporan praktikum ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca, khususnya bagi penulis sendiri. Penulis juga mengharapkan kritik dan saran yang membangun demi perbaikan laporan ini di masa yang akan datang.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat bagi semua pihak yang membacanya.

Buketrata,…… 2026

M. Syauqi Aufa Izzazullah

DAFTAR ISI

DAFTAR ISI .....
PENDAHULUAN.....
1.1 Latar Belakang.....
1.2 Rumusan Masalah.....
1.3 Tujuan Praktikum.....
BAB II DASAR TEORI.....
2.1 Function.....
2.2 Arrow Function & Callback.....
2.3 Array Dasar.....
2.4 Method Iterasi Array.....
2.5 Rekursi.....
BAB III PEMBAHASAN.....
3.1 Persiapan Project.....
3.2 Mendefinisikan Function.....
3.3 Arrow Function & Callback.....
3.4 Membuat dan Memanipulasi Array.....
3.5 Menggunakan Method Iterasi.....
3.6 Menulis Function Rekursif.....
BAB IV Tugas Praktikum.....
4.1 Tugas 1: Sistem Nilai Mahasiswa.....
4.2 Tugas 2: Pangkat dan Palindrom Rekursif.....
BAB V PENUTUP.....
5.1 Kesimpulan.....

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang pesat menuntut mahasiswa untuk memiliki kemampuan dalam bidang pemrograman, khususnya dalam memahami konsep dasar struktur data dan algoritma. Salah satu bahasa pemrograman yang banyak digunakan saat ini adalah JavaScript, yang tidak hanya digunakan untuk pengembangan web, tetapi juga untuk pengolahan data dan pembuatan aplikasi berbasis server menggunakan Node.js.

Dalam pemrograman, pemahaman terhadap konsep function dan array merupakan hal yang sangat penting. Function digunakan untuk mengorganisasi kode agar lebih terstruktur, efisien, dan mudah digunakan kembali. Sementara itu, array digunakan sebagai struktur data untuk menyimpan dan mengelola sekumpulan nilai secara teratur. Selain itu, perkembangan sintaks modern seperti arrow function dan penggunaan callback menjadi bagian penting dalam penulisan kode JavaScript yang lebih ringkas dan fleksibel.

Lebih lanjut, pemanfaatan method pada array seperti forEach, map, filter, dan reduce memungkinkan proses pengolahan data dilakukan secara lebih efektif dan efisien. Di sisi lain, konsep rekursi juga menjadi salah satu teknik penting dalam pemrograman yang digunakan untuk menyelesaikan permasalahan secara berulang dengan pendekatan pemanggilan fungsi terhadap dirinya sendiri.

Oleh karena itu, melalui praktikum ini, mahasiswa diharapkan dapat memahami serta mengimplementasikan konsep function, array, method array, callback, dan rekursi dalam bahasa JavaScript. Dengan pemahaman tersebut, mahasiswa diharapkan mampu mengembangkan logika pemrograman yang lebih baik serta siap dalam menghadapi permasalahan yang lebih kompleks di bidang struktur data dan algoritma.

1.2 Rumusan Masalah

1. Bagaimana cara mendefinisikan dan menggunakan function dalam bahasa JavaScript, termasuk penggunaan parameter dan nilai kembalian (return value)?
2. Bagaimana memahami dan menerapkan konsep scope variabel dalam function?
3. Bagaimana penggunaan arrow function sebagai bentuk penulisan function yang lebih modern?
4. Bagaimana konsep callback diterapkan dalam pemrograman JavaScript?
5. Bagaimana cara membuat, mengakses, dan memanipulasi array dalam JavaScript?
6. Bagaimana penggunaan method array seperti forEach, map, filter, dan reduce dalam pengolahan data?
7. Bagaimana konsep rekursi dapat diterapkan untuk menyelesaikan suatu permasalahan dalam pemrograman?

1.3 Tujuan Praktikum

1. Mendefinisikan dan memanggil function dengan parameter dan nilai kembalian (return value)
2. Memahami konsep scope variabel di dalam dan di luar function
3. Menulis arrow function sebagai sintaks modern pengganti function biasa
4. Menulis arrow function sebagai sintaks modern pengganti function biasa
5. Membuat dan memanipulasi array menggunakan berbagai method bawaan
6. Menggunakan method iterasi array: forEach, map, filter, dan reduce
7. Memahami rekursi sebagai teknik function yang memanggil dirinya sendiri

BAB ll

DASAR TEORI

2.1 Function

Function adalah blok kode yang diberi nama dan dapat dipanggil (dieksekusi) kapan saja dan berulang kali. Bayangkan function seperti sebuah mesin atau resep — kamu mendefinisikannya sekali, lalu bisa menggunakannya berkali-kali tanpa perlu menulis ulang kodenya.

Mengapa function penting? Prinsip utama pemrograman adalah DRY (Don't Repeat Yourself). Daripada menulis kode yang sama di banyak tempat, cukup bungkus kode itu dalam sebuah function.

Anatomi sebuah function:

• Parameter — Variabel input yang diterima function dari luar. Didefinisikan di dalam kurung ().

• Return value — Nilai yang dikembalikan function setelah selesai dieksekusi, menggunakan kata kunci return.

• Scope — Variabel yang dideklarasikan di dalam function hanya bisa diakses di dalam function itu saja (local scope). Variabel di luar function disebut global scope.

Perlu diingat: setelah JavaScript menemukan kata kunci return, eksekusi function langsung berhenti. Kode di bawah return tidak akan pernah dijalankan.

2.2 Arrow Function & Callback

Arrow function adalah sintaks penulisan function yang lebih ringkas, diperkenalkan di ES6 (2015). Alih-alih menulis function nama(params) { ... }, kita bisa menulis const nama = (params) => { ... }.

Aturan penulisan arrow function:

▪ Jika hanya ada satu parameter, kurung bisa dihilangkan: x => x * 2

▪ Jika body hanya satu baris dan langsung return, kurung kurawal dan return bisa dihilangkan: (a, b) => a + b

▪ Jika body lebih dari satu baris, tetap gunakan kurung kurawal dan return.

Callback adalah sebuah function yang dikirimkan sebagai argumen ke function lain. Function penerima akan memanggil callback tersebut di waktu yang tepat. Ini adalah konsep yang sangat fundamental di JavaScript — hampir semua event handling, timer, dan method array menggunakan callback.

2.3 Array Dasar

Array adalah struktur data yang menyimpan kumpulan nilai dalam satu variabel, berurutan dan terindeks. Bayangkan array seperti sebuah rak buku bernomor — setiap buku (elemen) memiliki posisi (indeks) yang dimulai dari angka 0.

Contoh: const buah = ['Apel', 'Mangga', 'Jeruk']

▪ buah[0] → 'Apel' (indeks pertama = 0)

▪ buah[1] → 'Mangga' (indeks kedua = 1)

▪ buah[2] → 'Jeruk' (indeks ketiga = 2)

▪ buah.length → 3 (jumlah elemen)

Array di JavaScript bersifat dinamis — ukurannya bisa berubah sewaktu-waktu. Array juga bisa menyimpan tipe data yang berbeda-beda dalam satu array (meskipun praktik terbaik adalah menyimpan satu jenis data per array).

Method bawaan array yang sering digunakan untuk manipulasi:

▪ push(x) — tambah elemen x di akhir array

▪ pop() — hapus dan kembalikan elemen terakhir

▪ unshift(x) — tambah elemen x di awal array

▪ shift() — hapus dan kembalikan elemen pertama

- splice(i, n) — hapus n elemen mulai indeks i
- slice(a, b) — salin elemen dari indeks a sampai sebelum b (non-destructive)
- indexOf(x) — cari indeks elemen x (-1 jika tidak ditemukan)
- includes(x) — cek apakah x ada di array (mengembalikan boolean)

2.4 Method Iterasi Array

JavaScript menyediakan method iterasi array yang powerful. Method-method ini menerima sebuah callback function sebagai argumen, dan secara otomatis memanggil callback untuk setiap elemen array.

- `forEach(callback)` — Menjalankan callback untuk setiap elemen. Tidak mengembalikan nilai apapun. Digunakan hanya untuk efek samping (side effects) seperti mencetak atau menyimpan ke database.

- `map(callback)` — Membuat array baru dengan menerapkan callback ke setiap elemen. Array asli tidak berubah. Digunakan untuk mentransformasi setiap elemen.

- `filter(callback)` — Membuat array baru yang hanya berisi elemen yang memenuhi kondisi (callback mengembalikan true). Array asli tidak berubah. Digunakan untuk menyaring elemen.

- `reduce(callback, nilaiAwal)` — Mereduksi seluruh array menjadi satu nilai tunggal. Callback
  menerima dua argumen: accumulator (nilai yang terakumulasi) dan currentValue (elemen saat ini).

Ingat: map dan filter selalu menghasilkan array baru, sedangkan reduce menghasilkan satu nilai (angka, string, object, dll).

2.5 Rekursi

Rekursi adalah teknik di mana sebuah function memanggil dirinya sendiri untuk memecahkan masalah yang lebih kecil secara bertahap, sampai mencapai kondisi berhenti.

Setiap function rekursif harus memiliki dua bagian:

Base case (kondisi berhenti) — kondisi di mana function tidak lagi memanggil dirinya sendiri. Tanpa ini, rekursi akan berjalan selamanya dan menyebabkan error Maximum call stack exceeded.

Base case (kondisi berhenti) — kondisi di mana function tidak lagi memanggil dirinya sendiri. Tanpa ini, rekursi akan berjalan selamanya dan menyebabkan error Maximum call stack exceeded.

Contoh: Menghitung faktorial dari n (n!):

▪ 5! = 5 × 4! = 5 × 4 × 3! = 5 × 4 × 3 × 2! = 5 × 4 × 3 × 2 × 1! = 120

▪ Base case: faktorial(1) = 1

▪ Recursive case: faktorial(n) = n × faktorial(n-1)

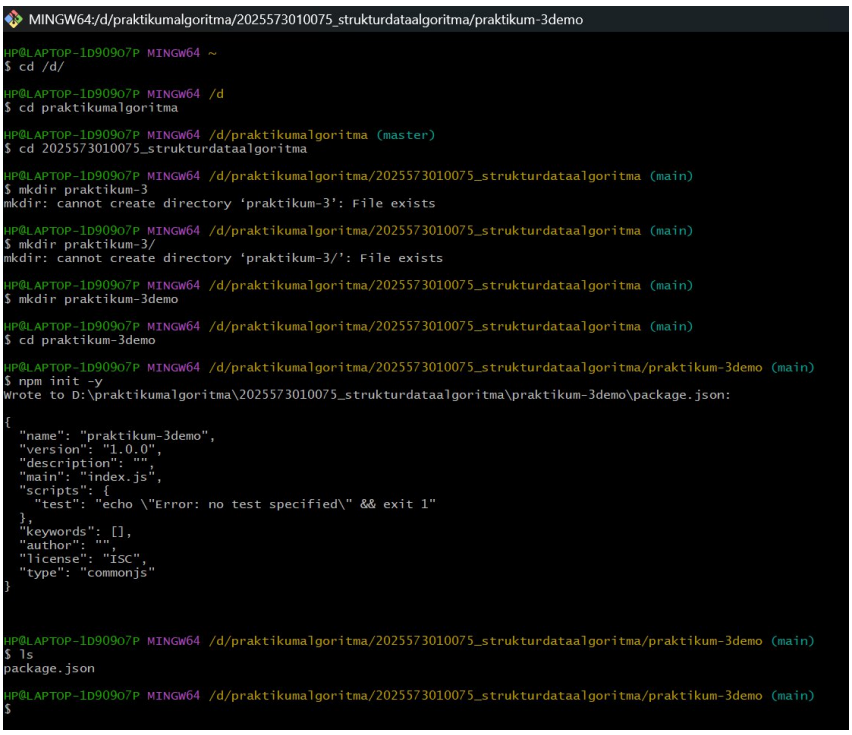
▪ 5! = 5 × 4! = 5 × 4 × 3! = 5 × 4 × 3 × 2! = 5 × 4 × 3 × 2 × 1! = 120 ▪ Base case: faktorial(1) = 1 ▪ Recursive case: faktorial(n) = n × faktorial(n-1)

BAB III

PEMBAHASAN

3.1 Persiapan Project

1. **Buat folder baru praktikum-3/ didalam repository Github dan inisialisasi npm project.**
2. **Buka Gitbash, navigasikan ke root folder repository kamu.**
   [cd nama-repository-kamu]
3. **Buat folder praktikum-3/ dan masuk kedalamnya.**
   [mkdir praktikum-3
   cd praktikum-3]
4. **Inisialisasi npm project.**
   [npm init -y]



Struktur Folder Yang diharapkan di akhir praktikum :

```
nama-repository/
└── praktikum-3/
    ├── package.json
    ├── 01-function-dasar.js
    ├── 02-arrow-callback.js
    ├── 03-array-dasar.js
    ├── 04-array-method.js
    ├── 05-rekursi.js
    └── tugas/
        ├── tugas-1.js
        └── tugas-2.js
```

3.2 Mendefinisikan Function

1. Di dalam folder praktikum-3/, buat file baru bernama 01-function-dasar.js.
 # Pastikan kamu berada di dalam folder praktikum-3
touch 01-function-dasar.js
2. Buka file 01-**function-dasar.js** di VS Code, lalu ketikkan kode berikut.

JS 01-function-dasar.js U ×
2025573010075_strukturdataalgoritma > praktikum-3 > JS 01-function-dasar.js > kali

```javascript
function sapa() {
console.log('halo! Selamat datang di praktikum javascript.');
}
sapa();
sapa();

function sapaNama(nama) {
    console.log(`Halo,${nama}! Selamat belajar.`);
}
sapaNama('Budi');
sapaNama('Siti');

function tambah(angka1, angka2){
    const hasil = angka1 + angka2;
    return hasil;
}

const hasilPenjumlahan = tambah(10, 25);
console.log('10 + 25 =', hasilPenjumlahan);
console.log('7 + 13 =', tambah(7,13));

function hitung(nilai, pengali = 2) {
    return nilai * pengali;
}

console.log(hitung(5));
console.log(hitung(5, 3));

const pesanGlobal = 'Saya ada dimana saja';

function cekScope() {
    const pesanLokal = 'Saya hanya ada di dalam function ini';
    console.log(pesanGlobal);
    console.log(pesanLokal);
}
cekScope();
console.log(pesanGlobal);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\praktikumalgoritma> cd 2025573010075_strukturdataalgoritma
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma> cd praktikum-3
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3> node 01-function-dasar.js
halo! Selamat datang di praktikum javascript.
halo! Selamat datang di praktikum javascript.
Halo,Budi! Selamat belajar.
Halo,Siti! Selamat belajar.
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada dimana saja
Saya hanya ada di dalam function ini
Saya ada dimana saja
```

3. Simpan file (Ctrl+S atau Cmd+S), kemudian jalankan file tersebut dari terminal.
node 01- function-dasar.js.

➢ Latihan

Latihan 1: Function Kalkulator Lengkap

1. Tambahkan kode baru di bawah kode yang sudah ada di file 01-function-dasar.js.
2. Buat function kurang(a, b) yang mengembalikan selisih a - b.
3. Buat function kali(a, b) yang mengembalikan hasil perkalian a * b.
4. Buat function bagi(a, b) yang mengembalikan hasil pembagian a / b. Tambahkan pengecekan: jika b === 0, tampilkan pesan 'Error: tidak bisa membagi dengan nol' dan return null.
5. Buat function kalkulator(a, operator, b) yang menerima 3 argumen. Gunakan if/else if atau switch untuk memanggil salah satu dari empat function di atas berdasarkan nilai operator ('+', '-', '*', '/').
6. Uji function kalkulator dengan setidaknya 5 pemanggilan berbeda, termasuk pembagian dengan nol.
7. Simpan dan jalankan: node 01-function-dasar.js.

```javascript
// Latihan 1 Function Kalkulator Lengkap

function tambah(a, b) {
    const hasil = a + b;
    return hasil;
}

function kurang(a, b) {
    const hasil = a - b;
    return hasil;
}

function kali(a, b) {
    const hasil = a * b;
    return hasil;
}

function bagi(a, b) {
    if (b === 0) {
        console.log('Error: tidak bisa membagi dengan nol');
        return null;
    }
    const hasil = a / b;
    return hasil;
}

function kalkulator(a, operator, b) {
    if (operator === '+') {
        return tambah(a, b);
    } else if (operator === '-') {
        return kurang(a, b);
    } else if (operator === '*') {
        return kali(a, b);
    } else if (operator === '/') {
        return bagi(a, b);
    } else {
        console.log('Operator tidak dikenali');
        return null;
    }
}

console.log('=== HASIL KALKULATOR ===');

console.log('10 + 5 =', kalkulator(10, '+', 5));
console.log('10 - 5 =', kalkulator(10, '-', 5));
console.log('10 * 5 =', kalkulator(10, '*', 5));
console.log('10 / 5 =', kalkulator(10, '/', 5));
console.log('10 / 0 =', kalkulator(10, '/', 0));
```

```
=== HASIL KALKULATOR ===
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2
Error: tidak bisa membagi dengan nol
10 / 0 = null
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3>
```

3.3 Arrow Function & Callback

1. Di dalam folder praktikum-3/, buat file baru bernama 02-arrow-callback.js

 # Pastikan kamu berada di dalam folder praktikum-3

touch 02-arrow-callback.js

2. Buka file 02-arrow-callback.js di VS Code, lalu ketikkan kode berikut.

2025573010075_strukturdataalgoritma > praktikum-3 > JS 02-arrow-callback.js > ...

```javascript
function kuadratBiasa(x) {
    return x * x;
}

const kuadrat = (x) => {
    return x * x;
};

const kuadratRingkas = x => x * x;

console.log('=== Perbandingan Penulisan ===');
console.log('Biasa   :', kuadratBiasa(5));
console.log('Arrow   :', kuadrat(5));
console.log('Ringkas :', kuadratRingkas(5));

const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;

console.log('Luas 4x6 :', luas(4,6));
console.log(salam('Budi', 'Pagi'));

function LakukanOperasi(angka, operasiCallback) {
    console.log(`Angka awal: ${angka}`);
    const hasil = operasiCallback(angka);
    console.log(`Hasil setelah operasi: ${hasil}`);
}

console.log('\n=== Callback ===');

LakukanOperasi(7, kuadratRingkas);
LakukanOperasi(10, x => x + 100);
LakukanOperasi(20, function(x) {
    return x / 2;
});

console.log('\n=== setTimeout (Callback) ===');
console.log('Pesan 1: sebelum timer');

setTimeout(() => {
   console.log('pesan 3: ini dari dalam setTimeout (setelah 1 detik)');
}, 1000);

console.log('Pesan 2: setelah mendaftarkan timer');
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\praktikumalgoritma> cd 2025573010075_strukturdataalgoritma
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma> cd praktikum-3
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3> node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa   : 25
Arrow   : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: sebelum timer
Pesan 2: setelah mendaftarkan timer
```

3. Simpan file (Ctrl+S atau Cmd+S), kemudian jalankan file tersebut dari terminal.
node 02-arrow-callback.js

➢ Latihan

Latihan 2: Pipeline Transformasi Data

1. Tambahkan kode baru di bawah kode yang sudah ada di file 02-arrow-callback.js.
2. Buat arrow function keHurufBesar yang menerima sebuah string dan mengembalikan string tersebut dalam huruf kapital semua. (Gunakan method .toUpperCase()).
3. Buat arrow function tambahSeru yang menerima sebuah string dan mengembalikan string tersebut dengan tanda !!! di akhir.
4. Buat arrow function hitungKata yang menerima sebuah string dan mengembalikan jumlah kata di dalamnya. (Petunjuk: gunakan .split(' ').length).
5. Buat function prosesKalimat(kalimat, transformasiCallback) yang menerima sebuah kalimat dan sebuah callback, lalu menerapkan callback ke kalimat tersebut dan menampilkan hasilnya.
6. Uji prosesKalimat dengan ketiga arrow function di atas menggunakan kalimat 'belajar javascript itu menyenangkan'.
7. Simpan dan jalankan. Pastikan outputnya sesuai harapan.

```javascript
// Latihan 2 : Pipeline Transformasi data


const keHurufBesar = (teks) => {
    return teks.toUpperCase();
};

const tambahSeru = (teks) => {
    return teks + '!!!';
};

const hitungKata = (teks) => {
    return teks.split(' ').length;
};

function prosesKalimat(kalimat, callback) {
    console.log(`\nKalimat awal: ${kalimat}`);
    const hasil = callback(kalimat);
    console.log(`Hasil setelah proses: ${hasil}`);
}


console.log('\n=== Latihan 2: Callback ===');

const kalimat = 'belajar javascript itu menyenangkan';
prosesKalimat(kalimat, keHurufBesar);
prosesKalimat(kalimat, tambahSeru);
prosesKalimat(kalimat, hitungKata);
prosesKalimat(kalimat, (teks) => teks.split('').reverse().join(''));
```

```
=== Latihan 2: Callback ===

Kalimat awal: belajar javascript itu menyenangkan
Hasil setelah proses: BELAJAR JAVASCRIPT ITU MENYENANGKAN

Kalimat awal: belajar javascript itu menyenangkan
Hasil setelah proses: belajar javascript itu menyenangkan!!!

Kalimat awal: belajar javascript itu menyenangkan
Hasil setelah proses: 4

Kalimat awal: belajar javascript itu menyenangkan
Hasil setelah proses: nakgnaneynem uti tpircsavaj rajaleb
pesan 3: ini dari dalam setTimeout (setelah 1 detik)
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3>
```

3.4 Membuat Dan Memanipulasi Array

1. Di dalam folder praktikum-3/, buat file baru bernama 03-array-dasar.js

 # Pastikan kamu berada di dalam folder praktikum-3

touch 03-array-dasar.js

2. Buka file 03-array-dasar.js. di VS Code, lalu ketikkan kode berikut.

JS 03-array-dasar.js U

2025573010075_strukturdataalgoritma > praktikum-3 > JS 03-array-dasar.js > ...

```javascript
const mahasiswa = ['Budi', 'Siti', 'Ahmad', 'Rina'];
const nilai     = [85, 92, 78, 95, 88];

console.log('=== Array Awal ===');
console.log('Mahasiswa:', mahasiswa);
console.log('Nilai    :', nilai);
console.log('Jumlah Mahasiswa:', mahasiswa.length);

console.log('\n=== Akses Elemen ===');
console.log('Elemen pertama :', mahasiswa[0]);
console.log('Elemen ketiga  :', mahasiswa[2]);
console.log('Elemen terakhir:', mahasiswa[mahasiswa.length - 1]);

mahasiswa[1] = 'Siti Rahayu';
console.log('\nSetelah diubah:', mahasiswa);

console.log('\n=== Manipulasi Array ===');
mahasiswa.push('Doni');
console.log('Setelah push   :', mahasiswa);

mahasiswa.unshift('Zahra');
console.log('Setelah unshift :', mahasiswa);

const dihapusAkhir = mahasiswa.pop();
console.log('Dihapus (pop) :', dihapusAkhir);
console.log('Setelah pop   :', mahasiswa);

const dihapusAwal = mahasiswa.shift();
console.log('Dihapus (Shift) :', dihapusAwal);
console.log('Setelah shift   :', mahasiswa);

console.log('\n=== Pencarian === ');
console.log('Indeks Ahmad :', mahasiswa.indexOf('Ahmad'));
console.log('Ada Rina     :', mahasiswa.includes('Rina'));
console.log('Ada Budi     :', mahasiswa.includes('Budi'));

const sebagian = nilai.slice(1, 4);
console.log('\nNilai asli  :',  nilai);
console.log('Slice [1,4]   :',  sebagian);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\praktikumalgoritma> cd 2025573010075_strukturdataalgoritma
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma> cd praktikum-3
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3> node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai    : [ 85, 92, 78, 95, 88 ]
Jumlah Mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga  : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push   : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop   : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (Shift) : Zahra
Setelah shift   : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad : 2
Ada Rina     : true
Ada Budi     : true

Nilai asli  : [ 85, 92, 78, 95, 88 ]
Slice [1,4]   : [ 92, 78, 95 ]
```

3. Simpan file (Ctrl+S atau Cmd+S), kemudian jalankan file tersebut dari terminal.
node 03-array-dasar.js.

➢ Latihan

Latihan 3: Manajemen Daftar Belanja

1. Tambahkan kode baru di bawah kode yang sudah ada di file 03-array-dasar.js.
2. Buat array daftarBelanja berisi 5 item belanjaan.
3. Tampilkan seluruh isi daftar beserta nomor urutnya menggunakan for loop. Contoh output: 1. Beras, 2. Gula, dst.
4. Tambahkan 2 item baru di akhir daftar menggunakan push.
5. Hapus item pertama dari daftar menggunakan shift dan tampilkan item yang dihapus.
6. Cek apakah item 'Susu' ada di daftar menggunakan includes. Tampilkan hasilnya dengan keterangan yang jelas.
7. Tampilkan jumlah item akhir di daftar.

```javascript
// Latihan 3 Manajemen Daftar Belanja

const daftarBelanja = ['Beras', 'Gula', 'Minyak', 'Telur'];

console.log('\n=== Array Awal ===');
console.log(daftarBelanja);


console.log('\n=== Akses Elemen ===');
console.log('Item pertama:', daftarBelanja[0]);
console.log('Item terakhir:', daftarBelanja[daftarBelanja.length - 1]);

console.log('\n=== Daftar Belanja ===');
for (let i = 0; i < daftarBelanja.length; i++) {
    console.log(`${i + 1}. ${daftarBelanja[i]}`);
}

daftarBelanja.push('Susu');
daftarBelanja.push('Kopi');

console.log('\nSetelah push:');
console.log(daftarBelanja);

let itemHapus = daftarBelanja.shift();
console.log('\nItem dihapus:', itemHapus);
console.log('Setelah shift:', daftarBelanja);


console.log('\n=== Pengecekan ===');
console.log('Ada Susu?', daftarBelanja.includes('Susu'));

console.log('\nJumlah item:', daftarBelanja.length);
```

```
=== Array Awal ===
[ 'Beras', 'Gula', 'Minyak', 'Telur' ]

=== Akses Elemen ===
Item pertama: Beras
Item terakhir: Telur

=== Daftar Belanja ===
1. Beras
2. Gula
3. Minyak
4. Telur

Setelah push:
[ 'Beras', 'Gula', 'Minyak', 'Telur', 'Susu', 'Kopi' ]

Item dihapus: Beras
Setelah shift: [ 'Gula', 'Minyak', 'Telur', 'Susu', 'Kopi' ]

=== Pengecekan ===
Ada Susu? true

Jumlah item: 5
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3>
```

3.5 Menggunakan Method Iterasi

1. Di dalam folder praktikum-3/, buat file baru bernama 04-array-method.js
 # Pastikan kamu berada di dalam folder praktikum-3
touch 04-array-method.js
2. Buka file 04-array-method.js di VS Code, lalu ketikkan kode berikut.

```js
const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];

console.log('=== forEach: Tampilkan Semua Nilai ===');
nilaiMahasiswa.forEach((nilai, indeks) => {
    console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});

const gradeHuruf = nilaiMahasiswa.map(nilai => {
    if (nilai >= 90) return 'A';
    if (nilai >= 80) return 'B';
    if (nilai >= 70) return 'C';
    if (nilai >= 60) return 'D';
    return 'E';
});

console.log('\n=== map: Nilai ke Grade ===');
console.log('Nilai :', nilaiMahasiswa);
console.log('Grade :', gradeHuruf);

const nilailulus = nilaiMahasiswa.filter(nilai => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter(nilai => nilai < 60);

console.log('\n=== Filter: Lulus dan Tidak Lulus ===');
console.log('Semua nilai :', nilaiMahasiswa);
console.log('Nilai lulus :', nilailulus);
console.log('Nilai Gagal :', nilaiGagal);

const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
    return akumulator + nilai;
}, 0);

const rataRata = totalNilai / nilaiMahasiswa.length;

console.log('\n=== reduce: Statistik Nilai ===');
console.log('Total nilai :', totalNilai);
console.log('Rata-rata   :', rataRata.toFixed(2));

const rataRataNilaiLulus = nilaiMahasiswa.filter(nilai => nilai >= 60).reduce((acc, val, idx, arr) => {
    return acc + val / arr.length;
}, 0);

console.log('\n=== Method Chaining ===');
console.log('Rata-rata nilai lulus', rataRataNilaiLulus.toFixed(2));
```

```
PS D:\praktikumalgoritma> cd 2025573010075_strukturdataalgoritma
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma> cd praktikum-3
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3> node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

=== Filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai Gagal : [ 55, 48 ]

=== reduce: Statistik Nilai ===
Total nilai : 590
Rata-rata   : 73.75

=== Method Chaining ===
Rata-rata nilai lulus 81.17
```

3. Simpan file (Ctrl+S atau Cmd+S), kemudian jalankan file tersebut dari terminal. **node** 04-array-method.js.

➢ Latihan

Latihan 4: Analisis Data Produk

1. Tambahkan kode baru di bawah kode yang sudah ada di file 04-array-method.js.
2. Deklarasikan array of objects berikut:
3. const produk = [{ nama: 'Laptop', harga: 8500000, stok: 5 }, { nama: 'Mouse', harga: 150000, stok: 0 }, { nama: 'Keyboard', harga: 450000, stok: 12 }, { nama: 'Monitor', harga: 3200000, stok: 0 }, { nama: 'Headset', harga: 350000, stok: 8 }]
4. Gunakan `filter` untuk mendapatkan array produk yang stok-nya lebih dari 0 (tersedia). Tampilkan hasilnya.
5. Gunakan `map` pada array produk yang tersedia untuk menghasilkan array baru berisi hanya nama produk-nya saja.
6. Gunakan `reduce` untuk menghitung total nilai inventaris dari semua produk yang tersedia (harga * stok).
7. Gunakan `forEach` untuk menampilkan semua produk dalam format: [TERSEDIA] Laptop - Rp8.500.000 (5 unit) atau [HABIS] Mouse - Rp150.000.
8. Simpan dan jalankan.

04-array-method.js U

2025573010075_strukturdataalgoritma > praktikum-3 > 04-array-method.js > ...

```javascript
// Latihan 4 Analisis Data Produk

const produk = [
    { nama: 'Laptop', harga: 7000000, tersedia: true },
    { nama: 'Mouse', harga: 200000, tersedia: true },
    { nama: 'Keyboard', harga: 350000, tersedia: false },
    { nama: 'Monitor', harga: 1500000, tersedia: true },
    { nama: 'Printer', harga: 1200000, tersedia: false }
];

console.log('\n=== Latihan 4: Array Method ===');

// FILTER
const produkTersedia = produk.filter(item => item.tersedia);

console.log('\nProduk yang tersedia:');
console.log(produkTersedia);

// MAP
const namaProduk = produk.map(item => item.nama);

console.log('\nNama semua produk:');
console.log(namaProduk);

// REDUCE
const totalHarga = produk.reduce((total, item) => total + item.harga, 0);

console.log('\nTotal harga produk:');
console.log(totalHarga);

console.log('\nDaftar produk:');

produk.forEach((item, index) => {
    console.log(`${index + 1}. ${item.nama} - Rp${item.harga}`);
});
```

```
=== Latihan 4: Array Method ===

Produk yang tersedia:
[
  { nama: 'Laptop', harga: 7000000, tersedia: true },
  { nama: 'Mouse', harga: 200000, tersedia: true },
  { nama: 'Monitor', harga: 1500000, tersedia: true }
]

Nama semua produk:
[ 'Laptop', 'Mouse', 'Keyboard', 'Monitor', 'Printer' ]

Total harga produk:
10250000

Daftar produk:
1. Laptop - Rp7000000
2. Mouse - Rp200000
3. Keyboard - Rp350000
4. Monitor - Rp1500000
5. Printer - Rp1200000
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3>
```

3.6 Menulis Function Rekursif

1. Di dalam folder praktikum-3/, buat file baru bernama 05-rekursi.js
 # Pastikan kamu berada di dalam folder praktikum-3
touch 05-rekursi.js
2. Buka file 05-rekursi.js di VS Code, lalu ketikkan kode berikut.

JS 05-rekursi.js U

2025573010075_strukturdataalgoritma > praktikum-3 > JS 05-rekursi.js > ...

```javascript
function faktorial(n) {
    if (n <= 1) return 1;

    return n * faktorial(n - 1);
}

console.log('=== Faktorial ===');
console.log('0! =', faktorial(0));
console.log('1! =', faktorial(1));
console.log('5! =', faktorial(5));
console.log('7! =', faktorial(7));

function fibonacci(n) {

    if (n === 0) return 0;
    if (n === 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

console.log('\n=== Fibonacci ===');
for (let i = 0; i <= 8; i++) {
    process.stdout.write(fibonacci(i) + ' ');
}
console.log('');

function jumlahArray(arr, indeks = 0) {

    if (indeks === arr.length) return 0;
    return arr[indeks] + jumlahArray(arr, indeks + 1);
}

const angka = [3, 7, 2, 9, 5];

console.log('\n=== Jumlah Array dengan Rekursi ===');
console.log('Array  :', angka);
console.log('Jumlah :', jumlahArray(angka));

function countdown(n) {

    if (n < 0) {
        console.log('Selesai!');
        return;
    }
    console.log(n);

    countdown(n - 1);
}

console.log('\n=== Countdown Rekursif ===');
countdown(5);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\praktikumalgoritma> cd 2025573010075_strukturdataalgoritma
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma> cd praktikum-3
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3> node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array  : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3>
```

BAB IV

TUGAS PRAKTIKUM

4.1 Tugas 1 : Sistem Nilai Mahasiswa

JS tugas-1.js U

2025573010075_strukturdataalgoritma > praktikum-3 > tugas > JS tugas-1.js > filterLulus

```javascript
// TUGAS 1: SISTEM NILAI MAHASISWA

const dataMahasiswa = [
    { nama: 'Syauqi', nilai: 85 },
    { nama: 'Firman', nilai: 92 },
    { nama: 'Danish', nilai: 58 },
    { nama: 'Rajul', nilai: 76 },
    { nama: 'Alfarabi', nilai: 64 },
    { nama: 'Syahidan', nilai: 45 }
];

function hitungStatistik(arrMahasiswa) {
    const totalMahasiswa = arrMahasiswa.length;

    const hasil = arrMahasiswa.reduce((acc, mhs) => {
        acc.totalNilai += mhs.nilai;
        if (mhs.nilai > acc.tertinggi) acc.tertinggi = mhs.nilai;
        if (mhs.nilai < acc.terendah) acc.terendah = mhs.nilai;

        return acc;
    }, {
        totalNilai: 0,
        tertinggi: -Infinity,
        terendah: Infinity
    });

    return {
        total: totalMahasiswa,
        rataRata: hasil.totalNilai / totalMahasiswa,
        tertinggi: hasil.tertinggi,
        terendah: hasil.terendah
    };
}
```

```javascript
function filterLulus(arrMahasiswa, batasLulus) {
    return arrMahasiswa.filter(mhs => mhs.nilai >= batasLulus);
}

function tambahGrade(arrMahasiswa) {
    return arrMahasiswa.map(mhs => {
        let grade;

        if (mhs.nilai >= 90) grade = 'A';
        else if (mhs.nilai >= 80) grade = 'B';
        else if (mhs.nilai >= 70) grade = 'C';
        else if (mhs.nilai >= 60) grade = 'D';
        else grade = 'E';

        return { ...mhs, grade };
    });
}

console.log('=== DATA MAHASISWA ===');
dataMahasiswa.forEach((mhs, i) => {
    console.log(`${i + 1}. ${mhs.nama} - ${mhs.nilai}`);
});

const statistik = hitungStatistik(dataMahasiswa);

console.log('\n=== STATISTIK ===');
console.log(`Total Mahasiswa : ${statistik.total}`);
console.log(`Rata-rata       : ${statistik.rataRata.toFixed(2)}`);
console.log(`Nilai Tertinggi : ${statistik.tertinggi}`);
console.log(`Nilai Terendah  : ${statistik.terendah}`);

const lulus = filterLulus(dataMahasiswa, 60);

console.log('\n=== MAHASISWA LULUS (>= 60) ===');
lulus.forEach(mhs => {
    console.log(`${mhs.nama} - ${mhs.nilai}`);
});

const denganGrade = tambahGrade(dataMahasiswa);

console.log('\n=== DATA DENGAN GRADE ===');
denganGrade.forEach(mhs => {
    console.log(`${mhs.nama} - ${mhs.nilai} (Grade: ${mhs.grade})`);
});
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\praktikumalgoritma> cd 2025573010075_strukturdataalgoritma
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma> cd praktikum-3
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3> cd tugas
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3\tugas> node tugas-1.js
=== DATA MAHASISWA ===
1. Syauqi - 85
2. Firman - 92
3. Danish - 58
4. Rajul - 76
5. Alfarabi - 64
6. Syahidan - 45

=== STATISTIK ===
Total Mahasiswa : 6
Rata-rata       : 70.00
Nilai Tertinggi : 92
Nilai Terendah  : 45

=== MAHASISWA LULUS (>= 60) ===
Syauqi - 85
Firman - 92
Rajul - 76
Alfarabi - 64

=== DATA DENGAN GRADE ===
Syauqi - 85 (Grade: B)
Firman - 92 (Grade: A)
Danish - 58 (Grade: E)
Rajul - 76 (Grade: C)
Alfarabi - 64 (Grade: D)
Syahidan - 45 (Grade: E)
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3\tugas>
```

4.2 Tugas 2 : Pangkat Dan Palindrom Rekursif

JS tugas-2.js U ×

2025573010075_strukturdataalgoritma > praktikum-3 > tugas > JS tugas-2.js > ...

```javascript
// TUGAS 2: PANGKAT DAN PALINDROM REKURSIF

function pangkat(basis, eksponen) {

    if (eksponen === 0) return 1;

    return basis * pangkat(basis, eksponen - 1);
}

function balikString(str) {

    if (str.length <= 1) return str;

    return str[str.length - 1] + balikString(str.slice(0, str.length - 1));
}

function cekPalindrom(str) {
    const hasilBalik = balikString(str);
    return str === hasilBalik;
}

console.log('=== PANGKAT REKURSIF ===');
console.log('2^3 =', pangkat(2, 3));
console.log('5^2 =', pangkat(5, 2));
console.log('7^0 =', pangkat(7, 0));
console.log('3^4 =', pangkat(3, 4));

console.log('\n=== BALIK STRING ===');
console.log('halo ->', balikString('halo'));
console.log('dunia ->', balikString('dunia'));
console.log('javascript ->', balikString('javascript'));

console.log('\n=== CEK PALINDROM ===');
const kata = ['katak', 'civic', 'level', 'halo'];

kata.forEach(k => {
    if (cekPalindrom(k)) {
        console.log(`${k} adalah palindrom`);
    } else {
        console.log(`${k} bukan palindrom`);
    }
});
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\praktikumalgoritma> cd 2025573010075_strukturdataalgoritma
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma> cd praktikum-3
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3> cd tugas
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3\tugas> node tugas-2.js
=== PANGKAT REKURSIF ===
2^3 = 8
5^2 = 25
7^0 = 1
3^4 = 81

=== BALIK STRING ===
halo -> olah
dunia -> ainud
javascript -> tpircsavaj

=== CEK PALINDROM ===
katak adalah palindrom
civic adalah palindrom
level adalah palindrom
halo bukan palindrom
PS D:\praktikumalgoritma\2025573010075_strukturdataalgoritma\praktikum-3\tugas>
```

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa pemahaman terhadap konsep function dan array dalam JavaScript sangat penting dalam pengembangan logika pemrograman. Function memungkinkan penulisan kode yang lebih terstruktur, efisien, dan dapat digunakan kembali, sedangkan array mempermudah pengelolaan sekumpulan data dalam satu variabel.

Selain itu, penggunaan arrow function memberikan cara penulisan yang lebih ringkas dan modern, serta callback memungkinkan fungsi digunakan secara dinamis sebagai parameter dalam fungsi lain. Method array seperti forEach, map, filter, dan reduce terbukti sangat membantu dalam proses pengolahan data secara efektif tanpa mengubah data asli.

Lebih lanjut, konsep rekursi memberikan pendekatan alternatif dalam menyelesaikan permasalahan dengan cara memecah masalah menjadi bagian yang lebih kecil melalui pemanggilan fungsi secara berulang. Dengan memahami base case dan recursive case, rekursi dapat digunakan untuk menyelesaikan berbagai permasalahan seperti faktorial, deret Fibonacci, perpangkatan, serta pengecekan palindrom.

Dengan demikian, praktikum ini berhasil meningkatkan pemahaman mahasiswa dalam menerapkan konsep dasar struktur data dan algoritma menggunakan JavaScript, serta melatih kemampuan berpikir logis dan sistematis dalam menyelesaikan permasalahan pemrograman.